Raspberry Pi Activity: My Binary Addiction

In this activity, you will implement a one-bit binary adder using LEDs, resistors, and push-button switches. You will need the following items:

- Raspberry Pi B v3 with power adapter;
- LCD touchscreen;
- Keyboard and mouse;
- Breadboard;
- GPIO interface board with ribbon cable; and
- LEDs, resistors, switches, and jumper wires provided in your kit.

Regarding the electronic components, you will need the following:

- 2x red LEDs;
- 4x green LEDs;
- 4x blue LEDs;
- 2x push-button switches;
- 9x 220Ω resistors; and
- 9x jumper wires.

The adder

Recall the single-bit half adder shown in a previous lesson:

It takes two single-bit inputs, A and B, and produces two outputs, S (the sum bit) and C (the carry bit). The half adder has the following truth table:

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

The question is, how do we implement these logic gates in a programming language? We have seen that if-statements are related to logic gates. For example, we can evaluate if one condition *and* another are true. If and only if both are true will the entire condition be true (and the statements in the true part of the if-statement will be executed). Therefore, one way to implement the truth table for a half adder is as follows:

```
1: if A is 0 and B is 0
```

```
 2: then
 3:     S ← 0
 4:     C ← 0
 5: else
 6:     if A is 1 and B is 1
 7:     then
 8:         S ← 0
 9:         C ← 1
10:     else
11:         S ← 1
12:         C ← 0
13:     end
14: end
```

Notice that this basically handles each row of the truth table above. The first if-statement handles the first row of the truth table (when A and B are both 0), and sets S and C to 0. The second if-statement handles the last row of the truth table (when A and B are both 1), and sets S to 0 and C to 1. The last case (the else part of the second if-statement) handles the two middle rows of the truth table, where either A or B is 1 (but not both), and sets S to 1 and C to 0.

This is how we would implement a half adder in Scratch, for example. Most general purpose programming languages (like Python), however, allow bitwise operations. That is, they can take Boolean inputs (like A and B) and implement the logic of primitive gates (e.g., *and* and *or*). This is a much simpler way to implement the logic! Plus, it allows us to significantly reduce the amount of code required to implement the half adder. Recall that S is the output of A *xor* B, and C is the output of A *and* B:

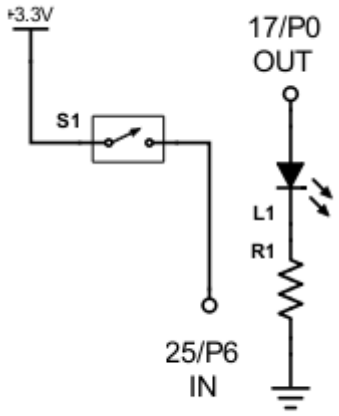
```
S = (~A & B) | (A & ~B)
C = A & B
```

That's it! Just two statements. Recall that Python support several bitwise operators, including *and* (&), *or* (|), and *not* (~). Also, the *xor* operation is performed as shown in an earlier lesson: *not* A *and* B *or* A *and not* B. Therefore, S is ultimately assigned the result of A *xor* B, and C is assigned the result of A *and* B. Although we structured our half adder such that the *xor* functionality was built using the three primitive gates (*and*, *or*, and *not*), Python has the *xor* bitwise operator (^) that we can use directly! This is quite useful:

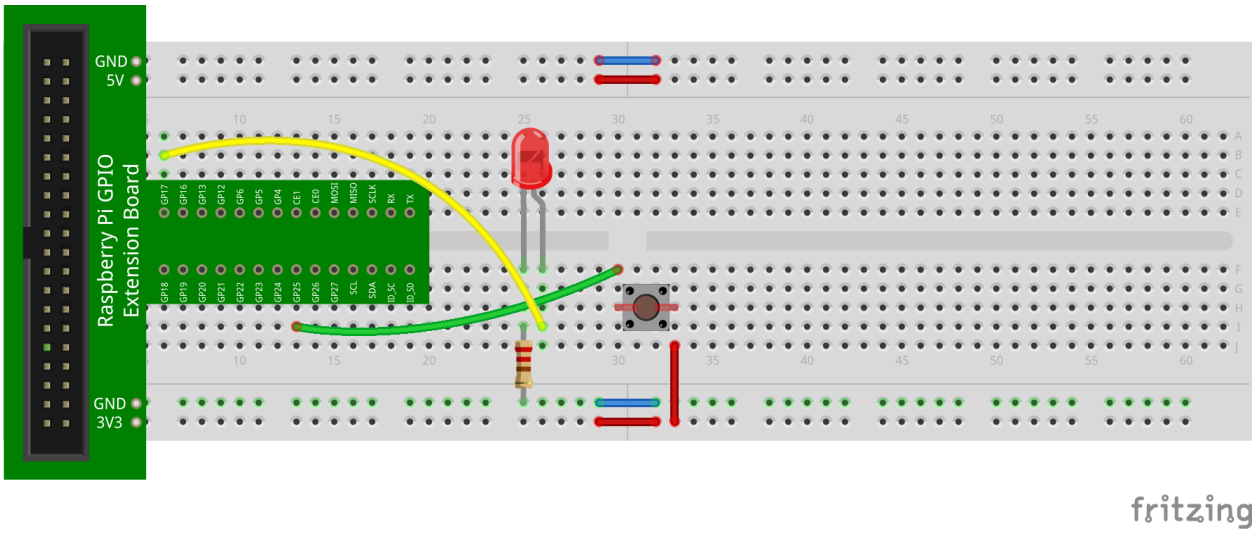
```
S = A ^ B
C = A & B
```

GPIO in Python

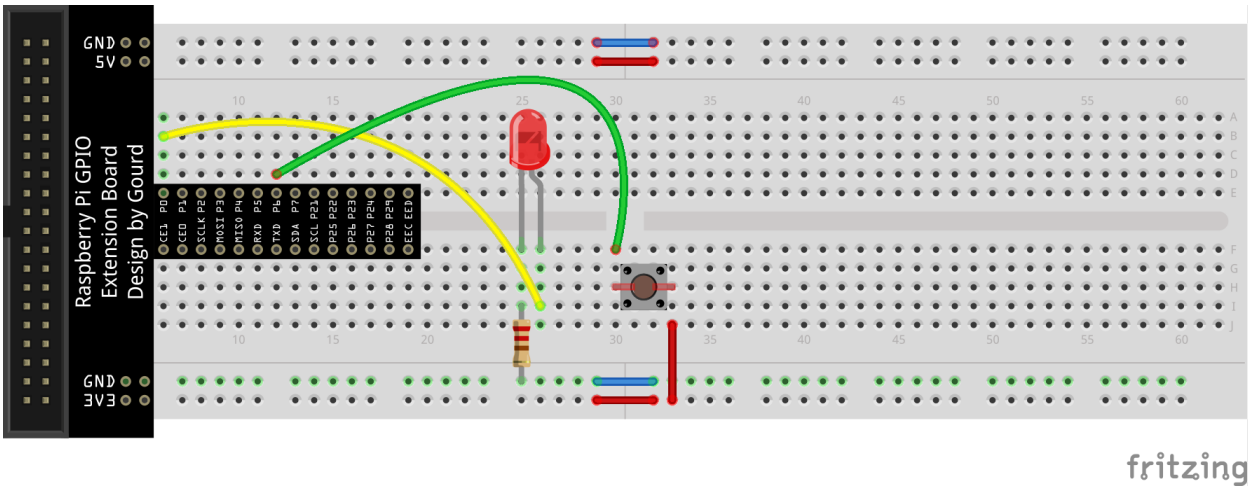
Before we continue, let's review how Python handles GPIO on the RPi. Implement the following single switch, single LED circuit:



Here's one way to layout this circuit:



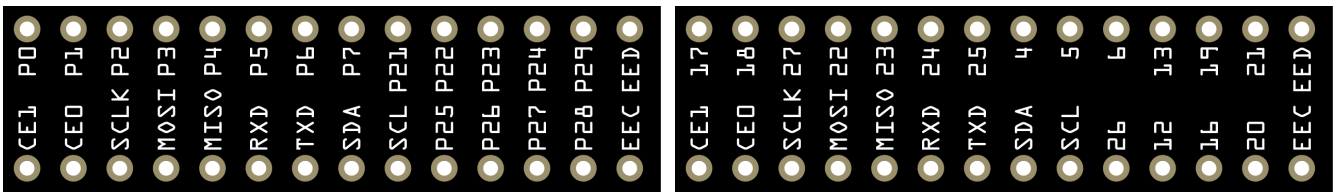
If you have the black GPIO interface board, layout the circuit as follows instead:



Recall that there are actually **three** different pin numbering schemes in use with GPIO pins on the RPi: (1) the **physical** pin order on the RPi; (2) the numbering assigned by the manufacturer of the **Broadcom** chip on the RPi; and (3) an older numbering assigned by an early RPi user who developed a library called **wiringPi**. Here's the cross-reference table shown in an earlier activity:

BCM	wPi	Name	Physical		Name	wPi	BCM
		3V3	1	2	5V		
2	8	SDA.1	3	4	5V		
3	9	SCL.1	5	6	GND		
4	7	GPIO.7	7	8	TXD	15	14
		GND	9	10	RXD	16	15
17	0	GPIO.0	11	12	GPIO.1	1	18
27	2	GPIO.2	13	14	GND		
22	3	GPIO.3	15	16	GPIO.4	4	23
		3V3	17	18	GPIO.5	5	24
10	12	MOSI	19	20	GND		
9	13	MISO	21	22	GPIO.6	6	25
11	14	SCLK	23	24	CE0	10	8
		GND	25	26	CE1	11	7
0	30	SDA.0	27	28	SCL.0	31	1
5	21	GPIO.21	29	30	GND		
6	22	GPIO.22	31	32	GPIO.26	26	12
13	23	GPIO.23	33	34	GND		
19	24	GPIO.24	35	36	GPIO.27	27	16
26	25	GPIO.25	37	38	GPIO.28	28	20
		GND	39	40	GPIO.29	29	21

If you have the green GPIO interface, you won't have to refer to the table since the RPi uses the BCM pin numbering scheme (which the green GPIO interface also uses). If you have the black GPIO interface, the following comparison of the GPIO interface boards labeled with both pin numbering schemes (shown in an earlier activity) will help:



In the layout diagram above, the LED is connected to **GP17** (which refers to BCM pin **17** on the RPi and **P0** on the black GPIO interface), and the switch is connected to **GP25** (which refers to BCM pin **25** on the RPi and **P6** on the black GPIO interface).

The goal is to detect a switch press by ensuring that the input pin to which it is connected is initially pulled down. When the switch is pressed, current flows from +3.3V to the input pin, which can be detected. The LED is then driven high. Here's a Python program that implements this:

```
import RPi.GPIO as GPIO
from time import sleep

# set the LED and switch pin numbers
led = 17
```

```
button = 25

# use the Broadcom pin mode
GPIO.setmode(GPIO.BCM)

# setup the LED and switch pins
GPIO.setup(led, GPIO.OUT)
GPIO.setup(button, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

# do this forever
while (True):
     # light the LED when the switch is pressed...
     # ...turn it off otherwise
     if (GPIO.input(button) == GPIO.HIGH):
          GPIO.output(led, GPIO.HIGH)
     else:
          GPIO.output(led, GPIO.LOW)
     sleep(0.1)
```

To make things more interesting, let's blink an LED once every second (i.e., on for 0.5 second, off for 0.5 second) by default. If the switch is pressed, let's blink the LED faster, once every 0.5 second (i.e., on for 0.25 second, off for 0.25 second):

```
import RPi.GPIO as GPIO
from time import sleep

# set the LED and switch pin numbers
led = 17
button = 25

# use the Broadcom pin mode
GPIO.setmode(GPIO.BCM)

# setup the LED and switch pins
GPIO.setup(led, GPIO.OUT)
GPIO.setup(button, GPIO.IN, pull_up_down=GPIO.PUD_UP)

# we'll discuss this later, but the try-except construct allows
# us to detect when Ctrl+C is pressed so that we can reset the
# GPIO pins
try:
     # blink the LED forever
     while (True):
          # the delay is 0.5s if the switch is not pressed
          if (GPIO.input(button) == GPIO.HIGH):
               delay = 0.5
          # otherwise, it's 0.25s
          else:
               delay = 0.25
```

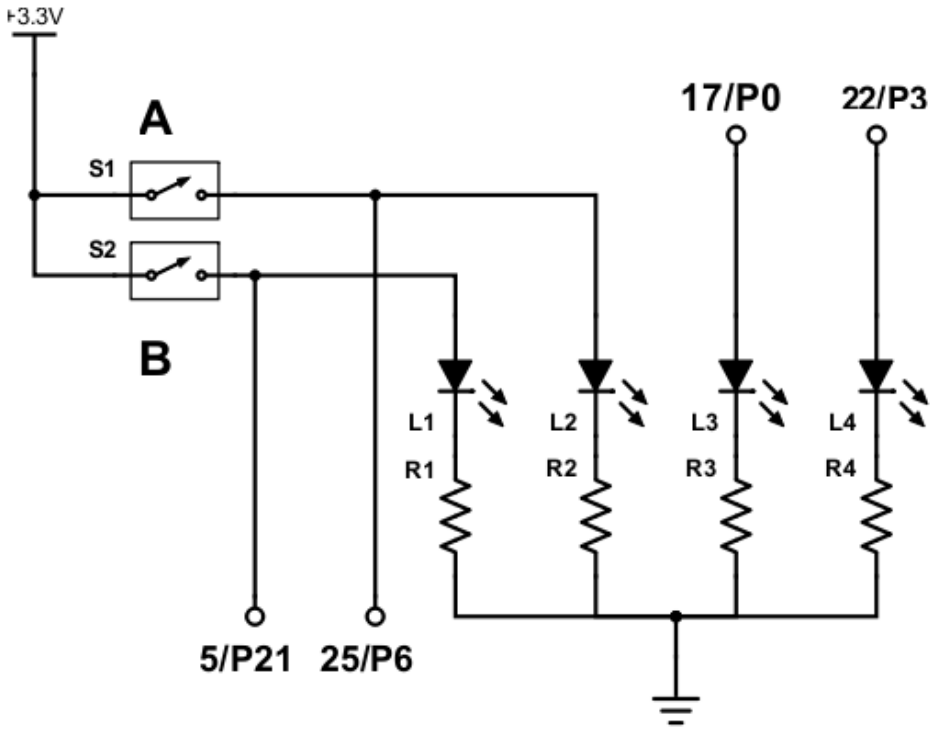
```
            # blink the LED
            GPIO.output(led, GPIO.HIGH)
            sleep(delay)
            GPIO.output(led, GPIO.LOW)
            sleep(delay)
# detect Ctrl+C
except KeyboardInterrupt:
     # reset the GPIO pins
     GPIO.cleanup()
```

You probably noticed the **try-except** construct. A comment notes that it will be discussed later (and it will!). For now, it's enough to know that such a construct is used to group statements that may cause an exception (i.e., some sort of abnormal event during runtime). In the case of the program above, the abnormal event is the user pressing Ctrl+C (which breaks out of the program). We can detect this and execute statements subsequently to do things like cleaning up and/or resetting the GPIO pins.

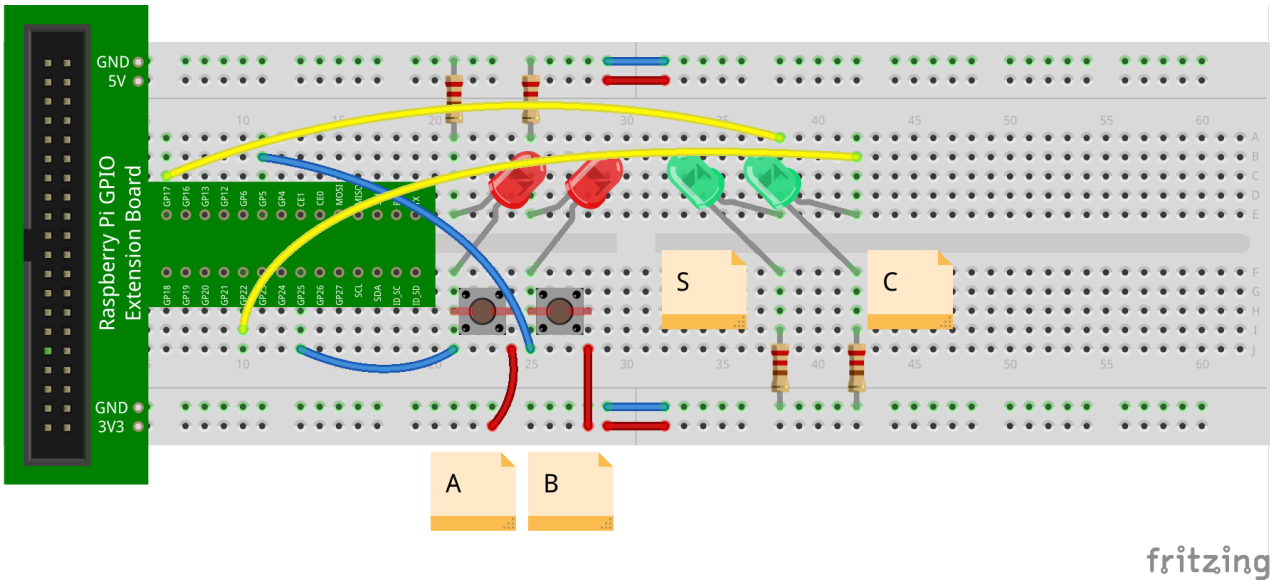
Now that GPIO on the RPi in Python has been reviewed, let's get to work on the circuit for this activity.

The circuit

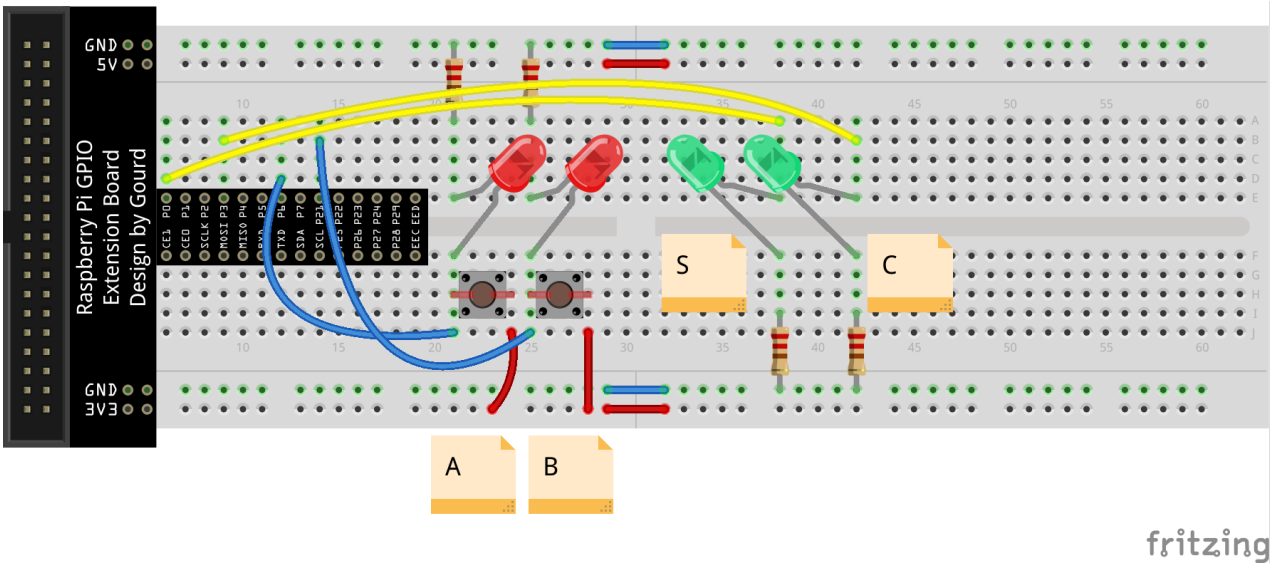
Implement the following half adder circuit. For this part of the activity, you will need two switches, four LEDs, four resistors, and some jumper wires:



Here's one way to layout this circuit:



If you have the black GPIO interface, layout the circuit as follows instead:



Note that the LEDs labeled S and C (the outputs) are green LEDs, and the LEDs labeled A and B (the inputs) are red LEDs. The input LEDs are also connected to the push-button switches. Since the switches are connected to +3.3V (i.e., they complete the circuit both to the input pins, GP25 (P6) and GP5 (P21), and to the red LEDs when closed), then the LEDs must be connected such that the anode (the longer positive side) is matched with the switch (i.e., the shorter negative side is connected to GND). This is illustrated in the circuit above. Pay close attention to polarity (i.e., where the negative and positive terminals of electronic components are) and wiring. Recall that the position of the resistor (either on the negative or positive side of the LED) doesn't matter. In the circuit above, the resistors are placed on the negative side of the LEDs.

Note that S (the green LED on the left) is connected to GP17 (P0), and C (the green LED on the right) is connected to GP22 (P3).

Note that the labels (A, B, S, and C) are strictly informative (i.e., they serve no function other than to provide situational awareness). It should be clear that the left push-button switch represents the bit input A, the right push-button switch represents the bit input B, the green LED on the left represents the bit output S, and the green LED on the right represents the bit output C. The red LEDs are wired to the push-button switches and provide feedback of the state of A and B (i.e., the left LED corresponds to the left push-button switch, and vice versa).

The code

Here's the entire program for the half adder in Python:

```
import RPi.GPIO as GPIO
from time import sleep

# set the GPIO pin numbers
inA = 25
inB = 5
outS = 17
outC = 22

# use the Broadcom pin mode
GPIO.setmode(GPIO.BCM)

# setup the input and output pins
GPIO.setup(inA, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
GPIO.setup(inB, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
GPIO.setup(outS, GPIO.OUT)
GPIO.setup(outC, GPIO.OUT)

# we'll discuss this later, but the try-except construct allows
# us to detect when Ctrl+C is pressed so that we can reset the
# GPIO pins
try:
     # keep going until the user presses Ctrl+C
     while (True):
          # initialize A, B, S, and C
          A = 0
          B = 0
          S = 0
          C = 0

          # set A and B depending on the switches
          if (GPIO.input(inA) == GPIO.HIGH):
               A = 1
          if (GPIO.input(inB) == GPIO.HIGH):
               B = 1

          # calculate S and C using A and B
          S = A ^ B      # A xor B
```

```
            C = A & B      # A and B

            # set the output pins appropriately
            # (to light the LEDs as appropriate)
            GPIO.output(outS, S)
            GPIO.output(outC, C)
    # detect Ctrl+C
    except KeyboardInterrupt:
         # reset the GPIO pins
         GPIO.cleanup()
```

After importing the required libraries, variables that map to GPIO pins are declared (*inA* representing the pin connected to switch A, *inB* representing the pin connected to switch B, *outS* representing the pin connected to LED S, and *outC* representing the pin connected to LED C). Next, the pins are setup (as either input or output pins). Since the switches are wired to +3.3V, the input pins are setup with a pull-down resistor. That is, their default value will be low at 0V. When a switch is pressed, this will bring up the input pin to 3.3V. Since the LEDs representing the inputs are also wired with the switches, they will light when the switches are pressed.

Next, the program detects the switch states and turns on the LEDs as appropriate. A and B are initialized to 0. If a switch is pressed, its corresponding input (A or B) is changed to 1. The values of S and C are then calculated (as A *xor* B for S, and A *and* B for C). Finally, the LEDs are triggered appropriately depending on the values of S and C.

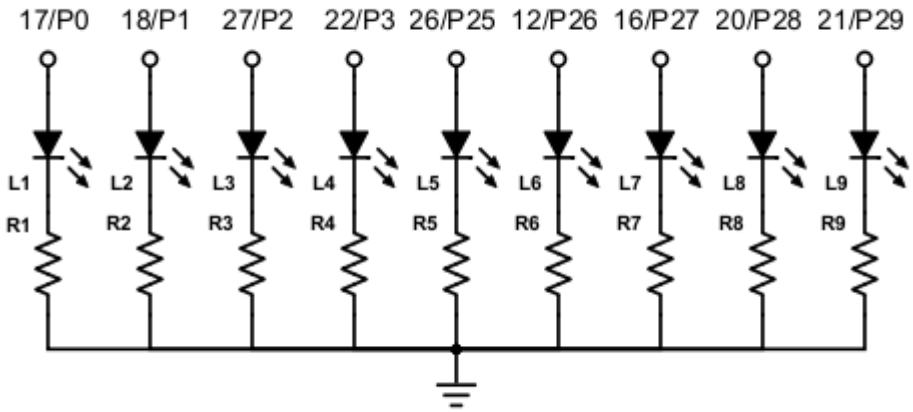
Perhaps a more efficient way to assign values for A and B is simply to modify the while loop as follows:

```
    while (True):
         # set A and B depending on the switches
         A = GPIO.input(inA)
         B = GPIO.input(inB)

         # calculate S and C depending on A and B
         S = A ^ B # A xor B
         C = A & B # A and B

         # set the output pins appropriately (to light the LEDs as
         #  appropriate)
         GPIO.output(outS, S)
         GPIO.output(outC, C)
```

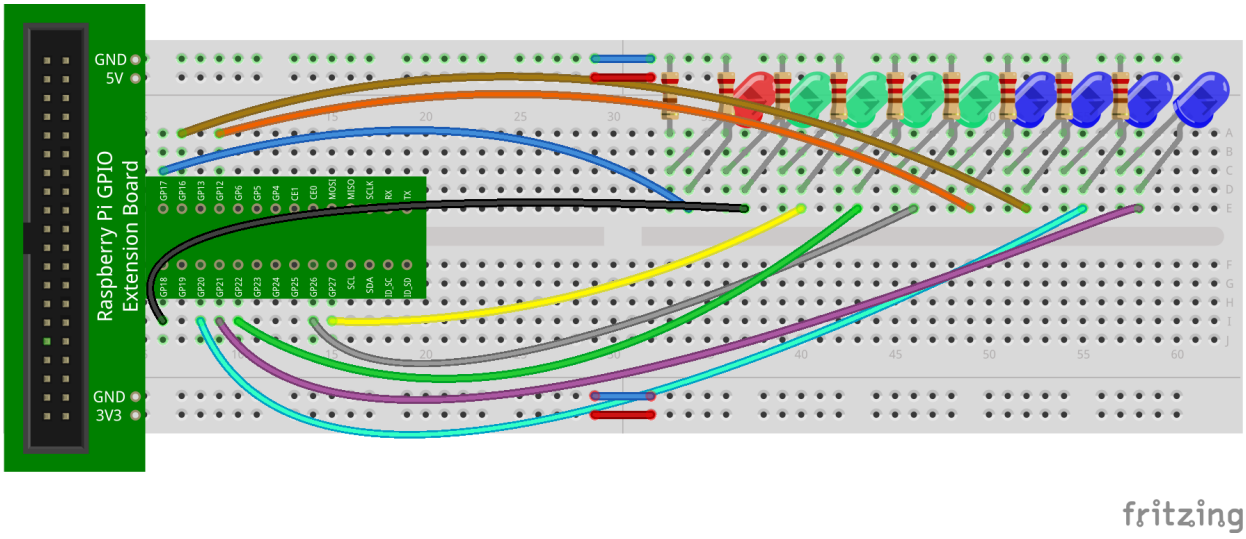
The end result is the same!

Extending this a bit

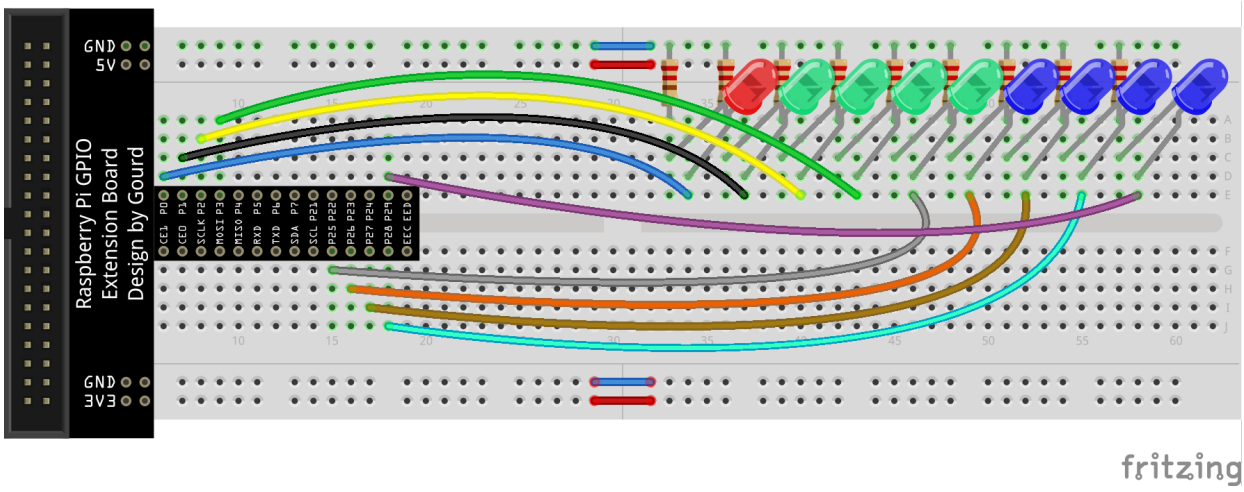
Take a look at the following circuit diagram. This time, there are nine LEDs, all connected to GPIO pins (GP17=P0, GP18=P1, GP27=P2, GP22=P3, GP26=P25, GP12=P26, GP16=P27, GP20=P28, GP21=P29) to a resistor that is connected to ground.



Here is one way to layout this circuit:



For the black GPIO interface, the layout diagram could look like this:



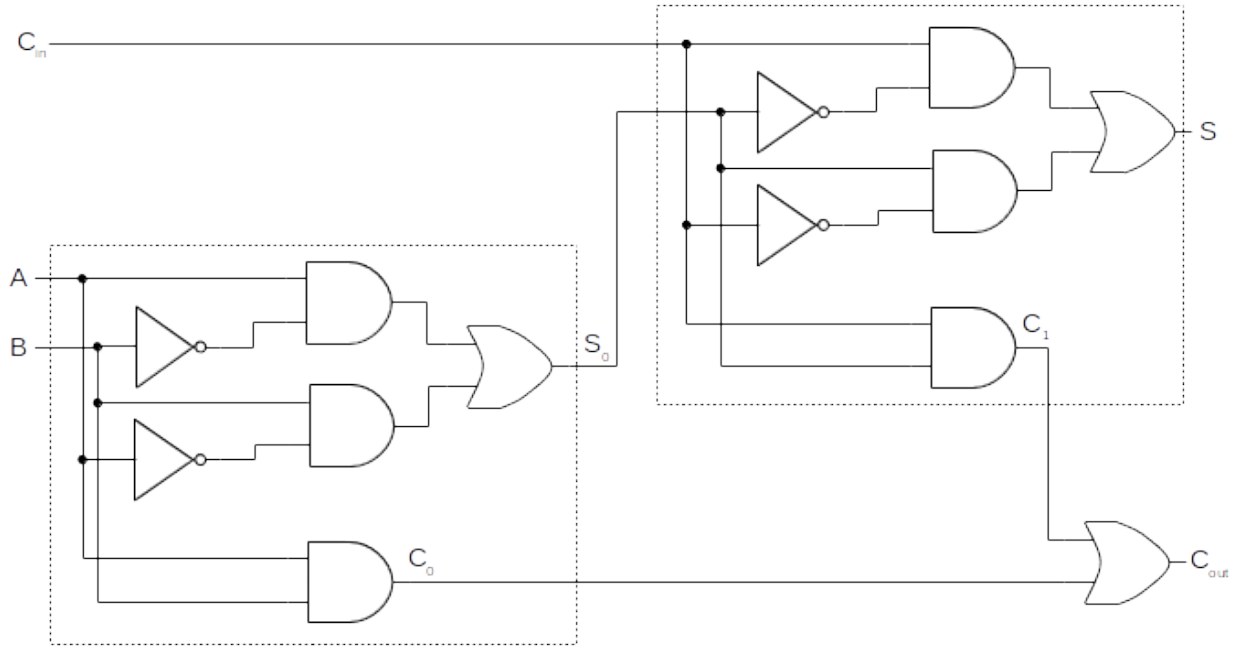
The LEDs represent the sum of two 8-bit numbers, with the least significant bit represented by the LED all the way to the right. For example, if the LEDs were to represent the sum of 94 + 113 = 207 (see the table below), then the state of the LEDs would be: off, on, on, off, off, on, on, on, on. The overflow bit (on the left) would be 0 (off).

	Binary										Decimal
	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
Carry		0	1	1	1	0	0	0	0		1
1st number			0	1	0	1	1	1	1	0	94
2nd number			0	1	1	1	0	0	0	1	113
Sum		0	1	1	0	0	1	1	1	1	207

If the LEDs were to represent the sum of 150 + 150 = 300 (see the table below), then the state of the LEDs would be: on, off, off, on, off, on, on, off, off. In this case, the overflow bit would be 1 (on).

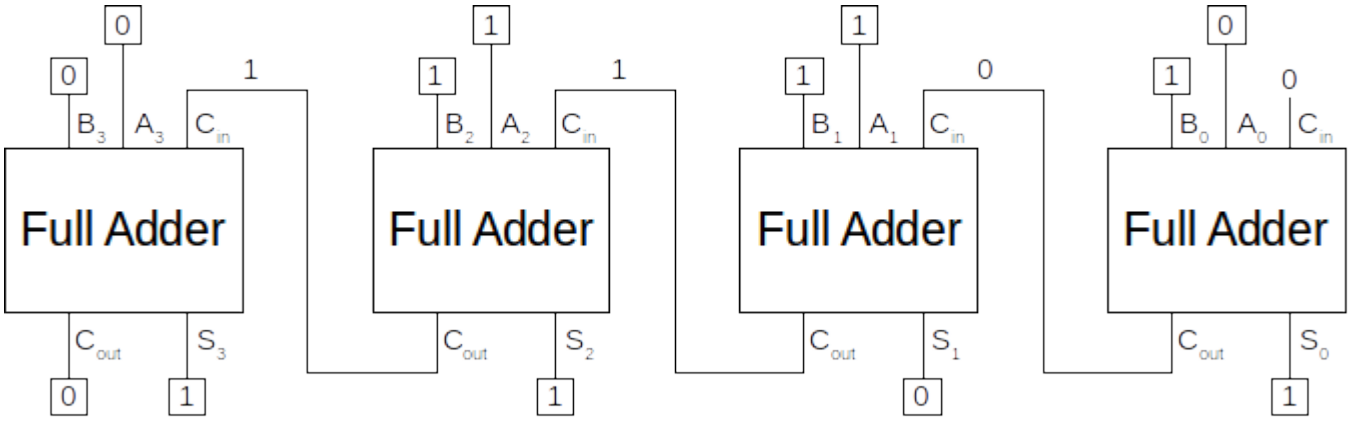
	Binary										Decimal
	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
Carry		1	0	0	1	0	1	1	0		1
1st number			1	0	0	1	0	1	1	0	150
2nd number			1	0	0	1	0	1	1	0	150
Sum		1	0	0	1	0	1	1	0	0	300

To make this work, you will need to implement a full adder as described in a previous lesson:



Recall that a full adder is made up of two half adders. One half adder computes the sum and carry of A and B. The sum is then brought into another half adder and added along with the carry in. The sum of this second half adder produces the actual sum of A and B plus the carry in. The carry out of this half adder is combined with the carry out of the first half adder through an *or* gate. The output is the carry out. You can take the script that implements the half adder (created in the first part of this activity) and extend it to a full adder.

Since each number is represented as a list, we will iterate through each, one bit at a time, and implement the full adder to produce a sum and carry out for each bit. Recall that the carry out is fed into the carry in for the next bit (to the left). We saw this when chaining full adders together to add two 4-bit numbers together:



In this activity, we are extending this to add two 8-bit numbers. The idea is the same.

Let's take a look at the beginning of the source code for this program. First, the header:

```
##################################################################
# Name:
# Date:
# Description:
##################################################################
import RPi.GPIO as GPIO       # bring in GPIO functionality
from random import randint    # to generate random integers
```

We'll make use of the `randint` function from the **random** library to generate the two random numbers.

Since there are many outputs, one for each LED, why don't we specify them all in a list. the following function sets up the GPIO pins for this program:

```
# function that defines the GPIO pins for the nine output LEDs
def setGPIO():
    # define the pins (change these if they are different)
    gpio = [17, 18, 27, 22, 26, 12, 16, 20, 21]
    # set them up as output pins
    GPIO.setup(gpio, GPIO.OUT)

    return gpio
```

Note how the pins are defined in a list called *gpio*. We then set each pin in the list to be an output pin.

The following snippet of code defines a function that generate a random 8-bit binary number:

```
# function that randomly generates an 8-bit binary number
def setNum():
    # create an empty list to represent the bits
    num = []
    # generate eight random bits
    for i in range(0, 8):
        # append a random bit (0 or 1) to the end of the list
        num.append(randint(0, 1))

    return num
```

This function first creates an empty list, called *num*. It then appends a random integer from 0 to 1, 8 times. The final number is then returned.

The following function handles turning on the appropriate LEDs representing the sum of the two 8-bit binary numbers:

```
# displays the sum (by turning on the appropriate LEDs)
def display():
    for i in range(len(sum)):
        # if the i-th bit is 1, then turn the i-th LED on
        if (sum[i] == 1):
            GPIO.output(gpio[i], GPIO.HIGH)
        # otherwise, turn it off
        else:
            GPIO.output(gpio[i], GPIO.LOW)
```

The function first iterates through the bits in the final sum. For each bit that is on (1), the matching GPIO pin is set high. For each bit that is off (0), the matching GPIO pin is set low.

And now we have reached the function that you are to implement in this activity – the full adder:

```
# function that implements a full adder using two half adders
# inputs are Cin, A, and B; outputs are S and Cout
# this is the function that you need to implement
def fullAdder(Cin, A, B):
    ##########################
    # write your code here!!!!!
    ##########################

    return S, Cout      # we can return more than one value!
```

Of course, you will need to implement this on your own! At then end of the function, two values are returned: S and Cout. This makes perfect sense, since that's the expected output of a full adder.

The following function controls the addition of each bit in the two 8-bit binary numbers. It effectively serves as the chain that connects the full adders (that you will implement in the function above):

```
# controls the addition of each 8-bit number to produce a sum
def calculate(num1, num2):
    Cout = 0            # the initial Cout is 0
    sum = []            # initialize the sum
    n = len(num1) - 1   # position of the right-most bit of num1

    # step through each bit, from right-to-left
    while (n >= 0):
        # isolate A and B (the current bits of num1 and num2)
        A = num1[n]
        B = num2[n]
        # set the Cin (as the previous half adder's Cout)
        Cin = Cout

        # call the fullAdder function that takes Cin, A, and...
        # ...B, and returns S and Cout
        S, Cout = fullAdder(Cin, A, B)

        # insert sum bit, S, at the beginning (index 0) of sum
        sum.insert(0, S)

        # go to the next bit position (to the left)
        n -= 1

    # insert the final carry out at the beginning of the sum
    sum.insert(0, Cout)

    return sum
```

Once all of the bits have been run through the full adder (and the sum has been completely calculated), the overflow bit of the sum (i.e., the left-most bit at the first position in the list *sum*) is set as the final C_{out}. This is why the circuit requires nine LEDs.

And now we have reached the main part of the program:

```
# use the Broadcom pin scheme
GPIO.setmode(GPIO.BCM)

# setup the GPIO pins
gpio = setGPIO()

# get a random num1 and display it to the console
num1 = setNum()
print(f" \t{num1}")
```

```
# get a random num2 and display it to the console
num2 = setNum()
print(f"+\t{num2}")

# calculate the sum of num1 + num2 and display it to the console
sum = calculate(num1, num2)
print(f"= {sum}”)

# turn on the appropriate LEDs to "display" the sum
display()

# wait for user input before cleaning up and resetting GPIO pins
input("Press ENTER to terminate")
GPIO.cleanup()
```

The main part of the program first sets the GPIO output pins (connected to the LEDs) by calling the function `setGPIO`. Again, this function defines a list that contains the pins corresponding to the nine output LEDs. It then iterates over them (via a for loop) and sets them up as output pins. The list is then returned to the main part of the program (note that the variable `gpio` contains this list).

Next, the first number is generated by calling the function `setNum`. Again, this function iteratively builds a list of eight random bits. Once finished, the list is returned to the main part of the program (note that the variable `num1` contains this list). The same occurs for the second number. Note the use of the `randint` function. It is imported through the *random* library. Its format is `randint(x, y)`, where *x* and *y* are the lower and upper values specified by the interval [*x*, *y*] to select a random integer from. For example, `randint(5, 44)` selects a random integer from 5 to 44.

Next, the sum is calculated by calling the function `calculate` (passing in the two numbers as parameters). Again, this function serves as the 8-bit adder that chains together eight full adders. It cycles through the two numbers from right-to-left, each time (i.e., for each bit) calling the `fullAdder` function. This function is provided C_{in}, A, and B as input parameters. It implements a full adder (made up of two half adders) and calculates (and returns) values for S and C_{out}. **Your task is to implement the `fullAdder` function.**

Finally, the `display` function is called, which turns on the appropriate LEDs that correspond to the bits that are on (1) in the variable `sum`.

Homework: Full Adder...Reloaded

Create the *fullAdder* function that implements a full adder (that is made up of two half adders). Of course, you will also need to implement the program previously covered to test appropriately. **Make sure to use bitwise operators in your implementation of the full adder! A template is provided on the class web site.**

You are to submit your Python source code only (as a .py file) through the upload facility on the web site.